

Università degli Studi di Torino

Dipartimento di Informatica



Tesi di Laurea Triennale in Informatica

**Sviluppo di un'applicazione web per il
suggerimento di Punti di Interesse**

Relatore:

Noemi Mauro

Correlatori:

Fabio Ferrero

Angelo Geninatti Cossatin

Candidato:

Agnese Andriani

Anno Accademico 2024/2025

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Sommario

Il lavoro presentato in questa tesi riguarda lo sviluppo di un'applicazione web per il supporto a un sistema di narrazione interattiva geolocalizzata. L'obiettivo principale dell'applicazione è offrire un'esperienza di gioco che possa unire l'esplorazione del territorio all'apprendimento, favorendo così l'attività fisica e consentendo all'utente di vivere storie personalizzate basate su luoghi reali, preferenze e obiettivi individuali. L'applicazione consente di creare un account personale, di selezionare i propri generi narrativi preferiti e vari punti di interesse, fornendo così storie tematiche che si svolgono su una mappa realizzata attraverso l'integrazione con Google Maps API e funzionalità di geolocalizzazione. Durante il percorso, all'utente verranno presentati i vari punti narrativi che andranno pian piano a costruire la storia. Inoltre, potrà seguire la narrazione tramite un sistema di Text-to-Speech che accompagna vocalmente la storia. Sono stati sviluppati moduli per la gestione dei progressi personali, la visualizzazione di statistiche settimanali (chilometri percorsi, calorie bruciate, passi) e il monitoraggio di obiettivi generali proposti dall'applicazione. Dal punto di vista tecnico, il progetto è composto da (1) un front-end sviluppato in React Native e Expo Router, caratterizzato da un'interfaccia modulare e responsive; (2) un back-end realizzato in Flask (Python), con connessione a un database MySQL gestito tramite un container Docker; (3) un sistema di comunicazione RESTful tra client e server per lo scambio di dati relativi agli utenti, alle preferenze e ai contenuti narrativi.

Indice

1	Introduzione	1
1.1	Obiettivi della tesi	1
1.2	Struttura della tesi	3
2	Il Progetto e il Contesto di	
	Utilizzo	4
2.1	Contesto e bisogni	4
2.2	Tipologie di utenti	5
2.3	Altri progetti e iniziative esistenti	6
2.3.1	Pokémon Go	6
2.3.2	GuideXpress	7
2.3.3	izi.TRAVEL	7
3	Frontend: contenuti e	
	funzionalità del progetto	8
3.1	Registrazione e accesso	8
3.2	La Home e l'inizio della storia	10
3.3	I POI e il calcolo della distanza	14
3.4	Obiettivi e Area Personale	15
4	Tecnologie utilizzate	19
4.1	React Native	19
4.2	Expo Router	19
4.3	TypeScript	20
4.4	CSS in React Native	21

4.5	Flask	21
4.6	MySQL	22
4.7	Google Maps API	22
4.8	HTTP	22
4.9	JSON	23
4.10	Librerie aggiuntive	24
4.10.1	React Native Vector Icons	24
4.10.2	AsyncStorage	24
4.10.3	React Native Responsive Screen	24
4.10.4	Expo Speech e Web SpeechSynthesis API	25
5	Dettagli tecnici	26
5.1	Architettura software e modello SPA	26
5.2	Modello dell'utente	27
5.3	Formalizzazione delle preferenze	29
5.4	Organizzazione del codice	31
5.5	Codice rilevante	32
5.5.1	Costruzione e gestione dei chunk di testo	32
5.5.2	Lettura dei chunk tramite TTS	33
5.5.3	Costruzione del testo con descrizioni e domande (point- ts.tsx)	34
5.5.4	Attivazione automatica del TTS ai punti di interesse	36
5.6	Database	37
5.6.1	Le tabelle	37
5.6.2	Il funzionamento	38
6	Conclusioni e Sviluppi Futuri	40

1 Introduzione

1.1 Obiettivi della tesi

L'attività fisica ha da sempre rappresentato un elemento fondamentale per il benessere e la salute delle persone. Tuttavia, negli ultimi decenni, per molti praticare attività fisica non è frequente. Questa tendenza è stata causata da diversi fattori, in particolare dall'aumento della richiesta di professioni che portano i lavoratori ad essere sempre più sedentari e dall'utilizzo di mezzi pubblici o privati per spostamenti brevi.

Secondo l'Organizzazione mondiale della sanità (OMS), quasi un terzo degli adulti del mondo non raggiunge i livelli di attività fisica necessari a mantenere un buono stato di salute. In particolare, in totale sarebbero circa 1,4 miliardi gli adulti nel mondo a rischio di malattie croniche a causa di uno stile di vita sedentario [5].

A tal proposito, in questa tesi si mira ad analizzare e sviluppare un sistema di raccomandazione di punti di interesse (POI) utilizzando tecniche narrative ispirate ai racconti di Sherazade de *Le mille e una notte*. L'obiettivo è stimolare l'utente a compiere attività fisica attraverso l'esplorazione del territorio, integrando le raccomandazioni in una trama continua e dinamica che si adatta al comportamento e alle preferenze personali.

Questo metodo, che richiama esperienze di *gamification* come *Pokémon Go*, mira a trasformare l'esercizio fisico in un'esperienza piacevole e coinvolgente, guidando gli utenti attraverso sequenze di punti di interesse consigliati e quiz per mantenere alta l'attenzione.

Il contributo di questa tesi consiste nello sviluppo dell'interfaccia grafica e

dell'architettura di base dell'applicazione, che permettono di visualizzare gli itinerari, la storia associata, gli obiettivi da raggiungere e il profilo personale dell'utente con grafici relativi a calorie, passi e chilometri percorsi.

L'applicazione è stata sviluppata attraverso una pagina web che permette la visualizzazione della mappa con la collocazione dell'utente in tempo reale e, successivamente, la storia e gli obiettivi generati. La pagina permette sia di iniziare una nuova storia, sia di continuare o mettere in pausa una storia già iniziata, includendo anche un supporto audio, il quale si occupa di narrare la storia e gli obiettivi quando questi vengono raggiunti. È possibile rimuovere o attivare l'audio nella pagina stessa.

L'obiettivo è quello di consentire all'utente di avviare una narrazione che si sviluppa lungo un percorso a partire da una posizione iniziale e di seguirne lo svolgimento in maniera interattiva. La storia, infatti, viene presentata passo dopo passo: ogni nuova parte del racconto si sblocca esclusivamente al raggiungimento del punto di interesse. Ogni volta che l'utente desidera iniziare una nuova narrazione, gli viene chiesto il genere che desidera per la storia, il tempo a disposizione per la narrazione e si ha la possibilità di personalizzare l'itinerario inserendo tra i punti proposti durante l'avanzamento della narrazione luoghi di suo interesse.

Questi possono includere monumenti o sculture, ma anche spazi legati alla sua quotidianità, integrando anche le esigenze personali. In questo modo, la narrazione non è un elemento statico, ma diventa un processo attivo che permette movimento e scoperta.

Ogni volta che un utente termina una storia, vengono aggiornati i suoi dati riguardanti chilometri, calorie e passi relativi a sette giorni. Queste statistiche sono fondamentali per il raggiungimento degli obiettivi generali presenti in una delle pagine dell'applicazione. Gli obiettivi generali sono stati pensati per essere aggiornati in futuro, in modo tale che gli utenti vengano spronati all'utilizzo dell'applicazione nel tempo.

Il *frontend* consiste nella realizzazione di un'interfaccia grafica chiara ed accattivante nelle varie sezioni dell'applicazione ed include una mappa interattiva nella *home* e durante lo svolgimento della narrazione di una storia. La comunicazione tra *frontend* e *backend* viene gestita attraverso varie API.

Il *backend* riguarda la raccolta dei dati e l'aggiornamento del database durante le varie richieste fatte dall'utente: il database viene aggiornato ogni volta che un utente si registra o avanza con i propri obiettivi; questo dà la possibilità di tenere traccia dei suoi progressi.

1.2 Struttura della tesi

La tesi è organizzata in sei capitoli, ognuno dei quali affronta un aspetto diverso del lavoro svolto, spaziando dai dettagli implementativi, al pubblico a cui ci si rivolge e ad alcuni aspetti più tecnici. Nello specifico:

- **Capitolo 2:** vengono descritte nello specifico le tipologie di utenti a cui il sistema è destinato, i quali possono avere interessi eterogenei nell'utilizzo della piattaforma. Inoltre, vengono descritti:
 - il contesto in cui l'applicazione si colloca;
 - le necessità che l'applicazione deve soddisfare.
- **Capitolo 3:** vengono descritte le funzionalità del sistema in termini di ruoli che l'utente può ricoprire e delle varie azioni che può svolgere all'interno del sistema.
- **Capitolo 4:** vengono descritte le tecnologie usate per realizzare il sistema, tra cui i linguaggi di programmazione e le librerie di supporto.
- **Capitolo 5:** vengono descritti i dettagli tecnici; si discute dell'architettura del software descrivendo in maniera più approfondita il modello dell'utente e il modo in cui vengono strutturate e formalizzate le sue preferenze. A tal proposito, viene descritto il database su cui ci si appoggia e si include una breve descrizione dell'organizzazione del codice, mostrando anche frammenti di codice rilevanti.
- **Capitolo 6:** vengono tratte le conclusioni e vengono illustrati gli sviluppi futuri di questo progetto.

2 Il Progetto e il Contesto di Utilizzo

Come descritto nel Capitolo 1, il progetto nasce con l'intento di promuovere l'attività fisica attraverso la combinazione di storie personalizzate e punti di interesse, producendo a tal proposito un sistema semplice, intuitivo e facilmente accessibile, in grado di guidare l'utente lungo il percorso proposto durante la narrazione.

2.1 Contesto e bisogni

Oggi, il mondo delle applicazioni rivolte al movimento è in continua espansione, grazie soprattutto alla crescente consapevolezza dell'importanza di mantenere uno stile di vita attivo. Tuttavia, molte soluzioni disponibili si limitano al tracciamento di dati come passi, calorie o chilometri percorsi, senza coinvolgere l'utente con elementi narrativi.

Dal punto di vista dell'utilizzo nel campo turistico, la maggior parte delle risorse presenti consiste in audioguide interattive per città e musei, che tuttavia non offrono un livello di personalizzazione elevato o aspetti di *gamification*. In particolare, quest'ultima ha avuto un impatto determinante in questo settore, come nel caso di applicazioni come *Pokémon Go* [1], che hanno dimostrato come l'inserimento di elementi di gioco e obiettivi all'interno di un contesto realistico stimoli gli utenti a muoversi di più, trasformando l'attività fisica in un'esperienza piacevole anche per chi è meno predisposto a svolgere attività fisica.

Un ulteriore bisogno riguarda la possibilità di unire intrattenimento, benessere e cultura in un'unica piattaforma. In particolare, oggi sono poco presenti applicazioni che offrono narrazioni dinamiche, e il progetto generale mira a colmare questo vuoto. Tuttavia, il lavoro di questa tesi non si concentra sullo sviluppo di queste storie, ma sulla progettazione e implementazione di un'applicazione che possa permettere all'utente di interagire con il sistema.

Attraverso l'applicazione sviluppata, l'utente ha la possibilità di simulare la propria esperienza scegliendo i generi a cui è più interessato, come ad esempio avventura, mistero o fantasy, e integrando il percorso proposto con la propria quotidianità. In questo modo, i luoghi visitati non vengono percepiti solo come destinazioni, ma diventano parte integrante di un racconto.

2.2 Tipologie di utenti

Il progetto è rivolto a un pubblico eterogeneo, sia dal punto di vista degli interessi sia dal livello di attività fisica praticata. È quindi possibile individuare diverse tipologie di utenti:

- **Utenti orientati al benessere fisico:** l'applicazione può essere utilizzata da utenti che desiderano integrare alla propria attività fisica quotidiana, che si tratti di corsa o di lunghe camminate, una narrazione che renda il tempo trascorso più piacevole.
- **Utenti appassionati di storia e del patrimonio artistico:** individui interessati a scoprire la storia della propria città e del proprio territorio. L'applicazione costituisce uno strumento per arricchire il cammino con contenuti culturali.
- **Turisti:** utilizzatori occasionali che possono approfondire la visita di una città attraverso l'esperienza alternativa fornita dall'applicazione. Quest'ultima può rendere il percorso turistico più strutturato e schematico grazie alla presenza di punti e obiettivi.
- **Utenti attratti dalla gamification:** persone interessate principalmente alla componente ludica dell'applicazione. Il raggiungimento de-

gli obiettivi e la risoluzione di quiz possono costituire un elemento fondamentale per rendere l'attività fisica divertente.

- **Utenti occasionali:** persone che desiderano integrare abitudini salutari come passeggiate, ma che necessitano di stimoli per mantenere costanza e motivazione. Per loro, la visualizzazione di obiettivi e progressi rappresenta un supporto essenziale.

2.3 Altri progetti e iniziative esistenti

Negli ultimi anni si è assistito a una crescente diffusione di applicazioni e progetti finalizzati a promuovere l'attività fisica, l'esplorazione del territorio e il turismo culturale attraverso strumenti digitali. Ad oggi esistono diverse applicazioni che hanno come intento quello di favorire l'attività fisica, la scoperta del territorio e l'esplorazione turistica. Nel dettaglio verranno analizzate tre applicazioni significative: *Pokémon Go*, *GuideXpress* e *izi.TRAVEL*.

2.3.1 Pokémon Go

Lanciata da Niantic nel 2016, *Pokémon Go* è l'esempio più noto di applicazione che ha saputo combinare l'attività fisica con la *gamification*. È stata la stessa Niantic a promuovere iniziative che incoraggiassero le persone a muoversi, sfruttando le proprie tecnologie per coinvolgere comunità locali e promuovere la socialità. Molte iniziative mirano a favorire l'amicizia, la cura degli spazi pubblici e la valorizzazione dell'arte nelle città [25].

Il successo è misurabile anche in termini di numeri: si stima che siano stati percorsi oltre 38 miliardi di chilometri dai giocatori attraverso le attività legate alla piattaforma [25]. L'aspetto innovativo risiede nell'aver trasformato il movimento in un'attività ludica globale, comprendendo anche fasce di età diverse.

A differenza di applicazioni come *Pokémon Go*, che puntano principalmente sulla *gamification* per incentivare il movimento, l'applicazione sviluppata in questa tesi introduce un elemento distintivo: la possibilità di vivere

delle storie interattive mentre si esplora la città. Non si tratta soltanto di raggiungere punti, ma di essere immersi in una narrazione che può assumere generi diversi, dal fantasy al mistero, o di essere legata a interessi specifici, come film o serie TV.

2.3.2 GuideXpress

GuideXpress è un'applicazione pensata per visitare a piedi le città attraverso un accompagnatore virtuale: ogni itinerario e ogni offerta è progettato appositamente per l'utente e lo guida attraverso un'esperienza personalizzata, modificando i suggerimenti dopo ogni viaggio. Sono inclusi giochi, quiz, la possibilità di accumulare punti e sbloccare premi. Quest'app sostituisce una guida fisica ed è in grado di svelare i segreti più nascosti della città [17].

Tuttavia, questa applicazione mira a trattare le narrazioni esclusivamente da un punto di vista storico e artistico, senza offrire la possibilità di scegliere tra diversi generi narrativi, di esplorare storie legate a interessi specifici (ad esempio *Harry Potter*) o di inserire luoghi legati alla quotidianità dell'utente.

2.3.3 izi.TRAVEL

izi.TRAVEL è una piattaforma di *storytelling* turistico che fornisce audio-guide interattive per città, musei e percorsi tematici. Dal 2011, un team di innovatori olandesi ha come obiettivo quello di promuovere un modo innovativo e più coinvolgente di visitare città e musei, facendo scoprire le loro storie. L'idea di base è quella di arricchire gli itinerari con contenuti multimediali, offrendo al contempo la possibilità di creare una rete di utenti che possono interagire tra loro [19].

Il punto di forza della piattaforma è la capacità di integrare mappe, audio e informazioni storiche; tuttavia, rimane un sistema che non si adatta alle preferenze individuali a causa della mancanza di scelta tra i generi e interessi, risultando quindi meno coinvolgente rispetto all'approccio narrativo dinamico proposto in questa tesi.

3 Frontend: contenuti e funzionalità del progetto

In questo capitolo vengono descritte in dettaglio le funzionalità offerte dall'applicazione, con particolare attenzione all'interfaccia utente. L'obiettivo è quello di fornire una panoramica ad alto livello, mostrando come ogni azione influenzi il comportamento dell'applicazione. Le varie sezioni sono accompagnate da schermate esplicative, così da poter mostrare le principali azioni che un utente può svolgere.

3.1 Registrazione e accesso

La prima funzionalità implementata è l'autenticazione: l'utente ha la possibilità di registrarsi o di accedere all'applicazione. La prima schermata permette all'utente di decidere quale azione vorrà compiere (Figura 1), nel caso in cui volesse registrarsi gli sarà data la possibilità di creare un nuovo account inserendo e-mail, nome, username e password (Figura 2). Questi dati verranno salvati all'interno del database attraverso l'utilizzo di API (acronimo di Application Programming Interface), ovvero un insieme di regole e protocolli che permettono la condivisione di dati tra applicazioni software differenti [18]. In seguito, l'utente registrato potrà autenticarsi tramite e-mail e password (Figura 3). In caso di successo, l'utente verrà memorizzato come autenticato e i suoi dati resteranno in memoria anche dopo la chiusura dell'applicazione.

Una volta inserite le informazioni in fase di registrazione, è possibile indicare le proprie preferenze in termini di generi narrativi (Figura 4) e punti

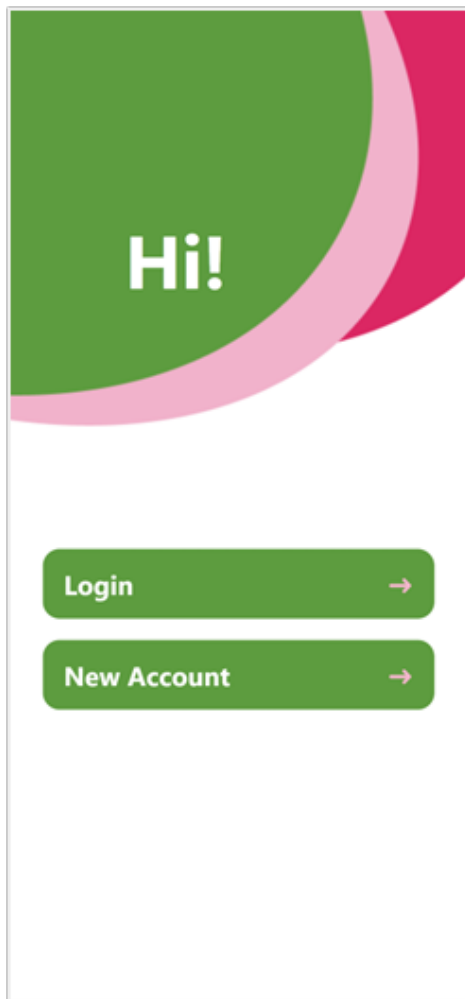


Figura 1: login e registrazione
utente

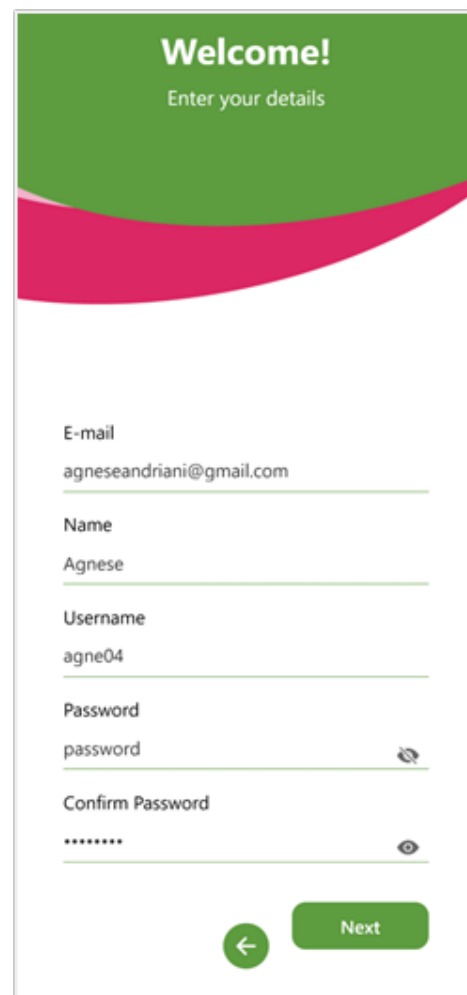


Figura 2: interfaccia registrazione
utente



Figura 3: login utente

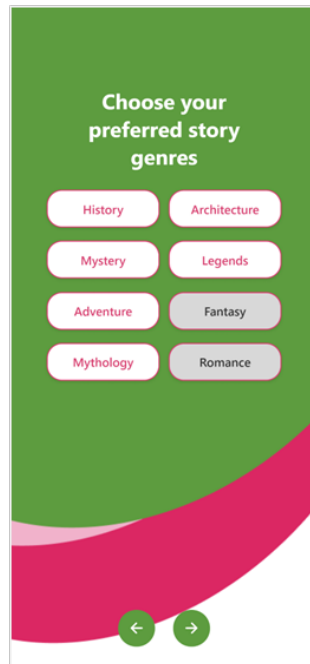


Figura 4: scelta dei generi

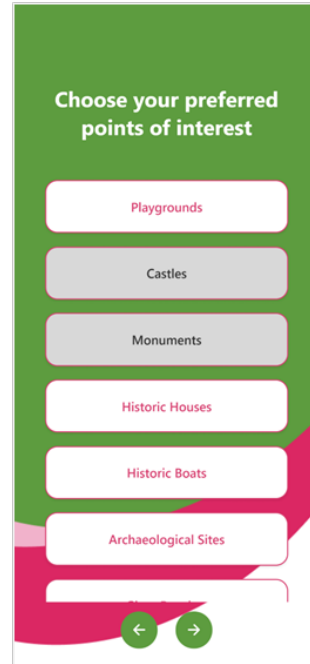


Figura 5: scelta dei POI

di interesse (Figura 5) che si desidera includere nelle narrazioni che verranno proposte, come ad esempio chiese, musei o castelli. Entrambi potranno essere modificati nella sezione Account dell'Area personale: queste scelte iniziali non sono permanenti e si adattano ai gusti dell'utente, rendendo l'app dinamica e sempre aggiornata. La possibilità di modificare le proprie preferenze costituisce una componente fondamentale nella progettazione dell'interfaccia dell'Area Personale dell'utente, poiché dimostra come l'applicazione sia in grado di adattarsi alle esigenze dell'utente. Premendo la freccia in avanti, l'utente crea ufficialmente il suo account e viene accolto nella Home, punto di partenza per le funzionalità principali dell'applicazione.

3.2 La Home e l'inizio della storia

Quando l'utente viene accolto nella Home, vengono reperiti i dati sulla sua geolocalizzazione: questo avviene grazie all'integrazione con Google Maps e

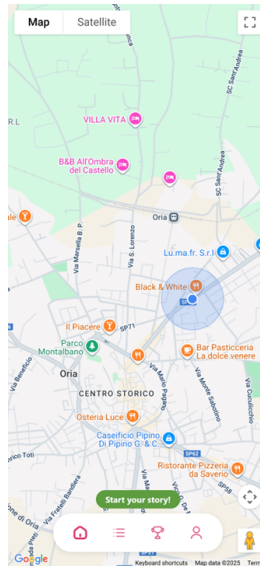


Figura 6: interfaccia Home

alle relative librerie che ci permettono di effettuare una ricerca dei luoghi. Inoltre, abbiamo la possibilità di aggiornare la posizione ogni volta che questa cambia, recuperando latitudine e longitudine della posizione del dispositivo e mantenendo un’accuratezza di 20 metri: questo cambierà la posizione del marker che rappresenta la posizione dell’utente (Figura 6).

La Home svolge la funzione di base da cui cominciare l’esperienza. Per poter navigare nelle varie sezioni dell’applicazione è presente una bottom navbar che include il tasto “Start your story!” da cui l’utente può avviare il processo di scelta di una nuova narrazione: gli verrà data la possibilità di selezionare il tempo a disposizione per la narrazione in ore e minuti utilizzando un TimePicker con scroll (Figura 7). Successivamente viene chiesto all’utente se desidera visitare un punto specifico (Figura 8): qui è possibile inserire punti a proprio piacimento o premere su “Skip” e saltare la scelta di un punto, dando libertà al sistema nella scelta della storia nel database. Infine, viene chiesto all’utente di selezionare il genere che preferisce utilizzare per la narrazione che gli verrà proposta (Figura 9): una volta indicate tutte le preferenze, il sistema recupererà dal database la storia che soddisfa tutte le richieste fatte dall’utente.

Oltre a “Start your story!”, la bottom navbar contiene anche altri elementi di navigazione: il tasto che fa tornare alla Home, quello che ci consente di accedere alla sezione dedicata ai punti da raggiungere durante la narrazione, quello dedicato alla pagina sugli obiettivi generali che l’utente deve raggiungere, come ad esempio completare la sua prima storia, e infine il tasto dedicato all’area personale, dove sono raccolte le informazioni e le preferenze dell’utente.



Figura 7: timepicker

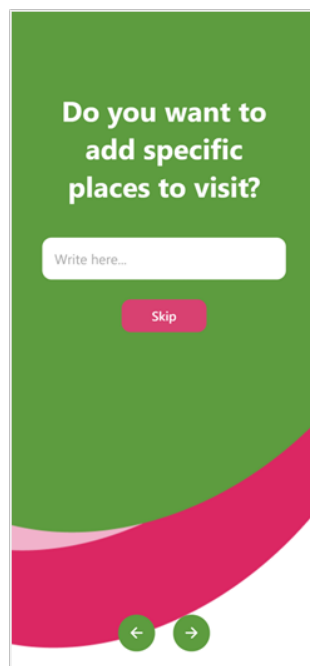


Figura 8: scelta POI

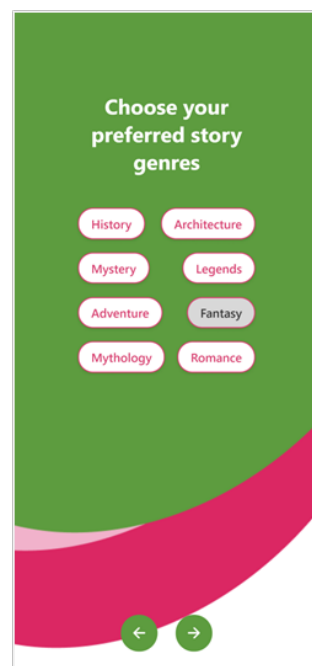


Figura 9: scelta genere

Recuperata la storia dal database, l’utente potrà dare inizio alla sua esperienza narrativa. Nel caso mostrato (Figure 7-8-9), all’utente verrà presentata una storia fantasy della durata di 30 minuti, senza vincoli legati al punto di interesse. La pagina principale (Figura 10) presenta la mappa con il punto di partenza dell’utente, il titolo, l’introduzione e la trama della storia: questi sono visibili all’interno di un componente scorrevole e comprimibile, che permette sia la lettura manuale sia l’ascolto tramite sintesi vocale. L’utente potrà dunque gestire la lettura, mettendola in pausa o silenziandola, attraverso gli appositi tasti. Un’altra funzionalità è quella di terminare la storia

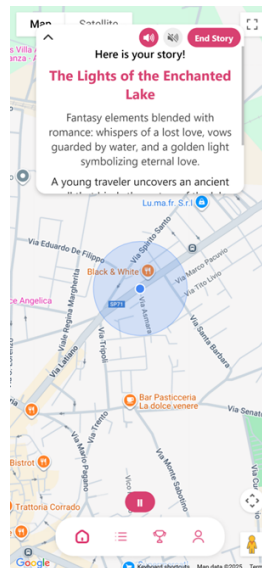


Figura 10: Home storia iniziata

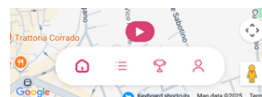


Figura 11: navbar storia iniziata

con il tasto “End Story”, il quale ci riporterà alla Home. Un elemento chiave è il cambiamento di stato visibile attraverso la bottom navbar. Quest’ultima cambierà dinamicamente il proprio stato in base allo stato della storia. Gli stati della navbar sono tre: start, stop e continue: lo stato start è abbinato alla navbar con bottone centrale verde con “Start your story!”, gli stati stop e continue sono relativi alla storia quando è in corso, essi ci permettono di mettere in pausa o di riprendere la narrazione nel caso in cui l’utente ne avesse bisogno. Si è deciso di abbinare tali stati a simboli che riconducessero al mondo della musica, dunque il triangolo per la pausa (Figura 11) e i due rettangoli verticali per lo stato di narrazione attiva (Figura 10). Queste operazioni influenzano anche la narrazione audio, la quale verrà messa in pausa quando si cliccherà sul tasto della navbar.

La posizione dell’utente viene aggiornata in tempo reale e ogni volta che la posizione cambia, vengono salvate latitudine e longitudine della nuova

posizione con un livello di accuratezza in metri, ovvero il raggio in incertezza della posizione.

3.3 I POI e il calcolo della distanza

L'accuratezza della posizione è fondamentale per il calcolo della distanza per il raggiungimento dei punti di interesse. La pagina dedicata ai punti di interesse raggiunti si popolerà solo nel momento in cui l'utente sarà in un raggio di al massimo 50 metri dal punto. Per calcolare la distanza tra la posizione dell'utente e il prossimo punto da raggiungere è stata utilizzata un'implementazione dell'algoritmo di Haversine, uno dei modi più diffusi per il calcolo della distanza tra due punti utilizzando la longitudine e la latitudine. Viene utilizzata la seguente formula:

$$\text{Haversine}(\theta) = \sin^2\left(\frac{\theta}{2}\right)$$

Questa equazione, dove φ rappresenta la latitudine, λ la longitudine e R il raggio terrestre (raggio medio = 6371), mostra come la formula venga adattata per includere le coordinate geografiche. In particolare, gli angoli devono essere espressi in radianti per poter essere utilizzati correttamente nelle formule trigonometriche [2].

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

Dove:

- φ_A, φ_B sono le latitudini dei due punti,

- λ_A, λ_B sono le longitudini dei due punti,
- R è il raggio della Terra,
- d rappresenta la distanza più breve tra due punti sulla superficie terrestre, calcolata lungo la sfera.

Quando l'utente si avvicina al punto di interesse, comparirà il testo relativo che gli consentirà di proseguire con la narrazione. La storia si sviluppa progressivamente man mano che l'utente raggiunge i vari punti. Ogni punto è arricchito con domande che spingono l'utente a prendere parte attiva al racconto. Oltre alla lettura sullo schermo, anche in questa sezione è possibile ascoltare la narrazione grazie alla sintesi vocale, la quale riproduce il testo del punto, la relativa domanda e le varie risposte a disposizione (Figura 12). Una volta selezionata la risposta che si ritiene corretta, si può confermare la scelta premendo su "Done", che farà comparire un alert per notificare l'utente se la risposta data è corretta o meno. Il testo del prossimo punto comparirà quando l'utente si avvicinerà alla nuova tappa.

3.4 Obiettivi e Area Personale

La pagina Goals è stata pensata per incentivare l'utente a utilizzare l'applicazione ponendogli degli obiettivi da raggiungere. I dati sugli obiettivi vengono aggiornati al termine di ogni narrazione (Figura 13). Nel nostro caso, il profilo era appena stato creato, perciò, terminando la storia avviata, abbiamo terminato la prima storia e i dati relativi ai passi e ai km sono stati aggiunti.

L'Area Personale è più strutturata ed è un altro spazio dedicato all'utente. È suddivisa in due sottosezioni: Rating e Settings. Nella sezione Rating, l'utente può visualizzare i propri progressi settimanali attraverso dei grafici, in particolare con istogrammi e un diagramma a linee che possono risultare immediati e facili da leggere. I parametri principali per l'asse delle ascisse in ogni grafico sono i giorni della settimana da lunedì a domenica mentre, per l'asse delle ordinate, valori che partono dallo zero e raggiungono valori

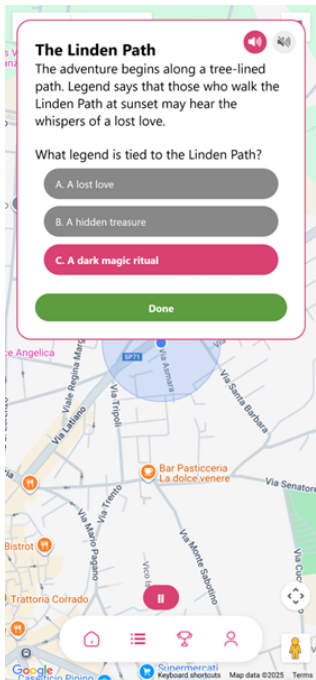


Figura 12: POI relativi alla storia

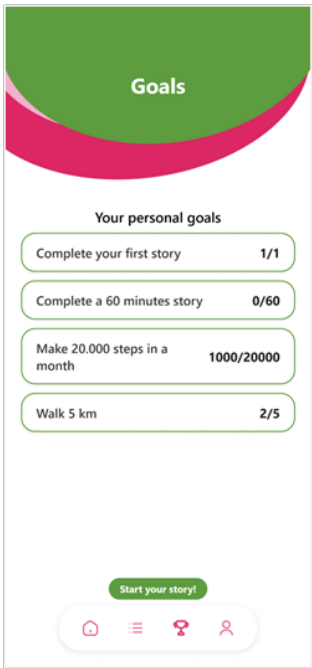


Figura 13: Goals

adeguati a ogni parametro che viene valutato (Figura 14-15). Nella sezione Settings, è uno spazio dedicato alla personalizzazione della propria esperienza d'uso (Figura 16). Tra le pagine implementate abbiamo il Logout, che ci riporta alla schermata di Accesso o Registrazione e l'Account, che costituisce la pagina con maggiore importanza all'interno della sezione. Nell'account l'utente può modificare le preferenze indicate in fase di registrazione, come i generi narrativi o i punti di interesse desiderati. La modifica di questi tratti dell'account ci riporta alla pagina in cui l'utente ha dovuto scegliere le preferenze in fase di registrazione, offrendogli un ambiente a lui già familiare. Inoltre, può visualizzare i dati personali come e-mail, nome e password per avere un quadro generale del proprio profilo (Figura 17).

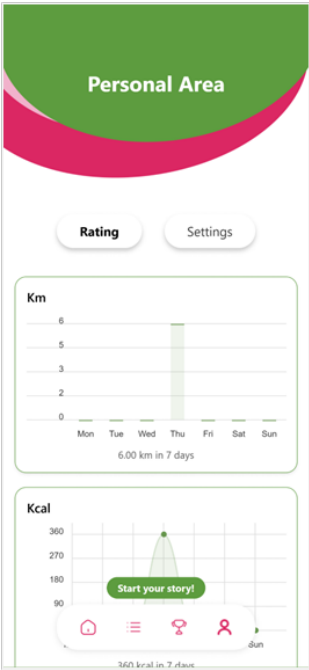


Figura 14: rating 1

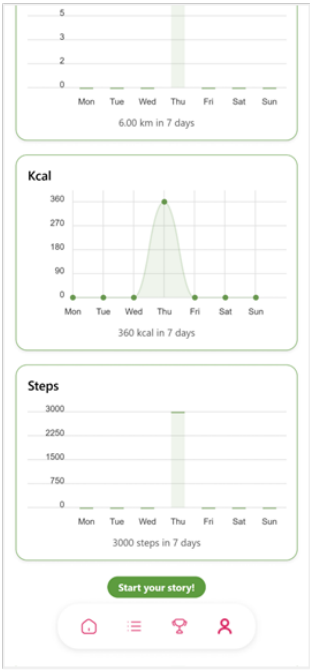


Figura 15: rating 2

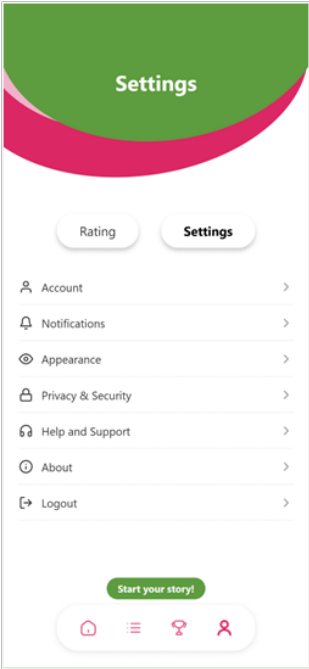


Figura 16: settings

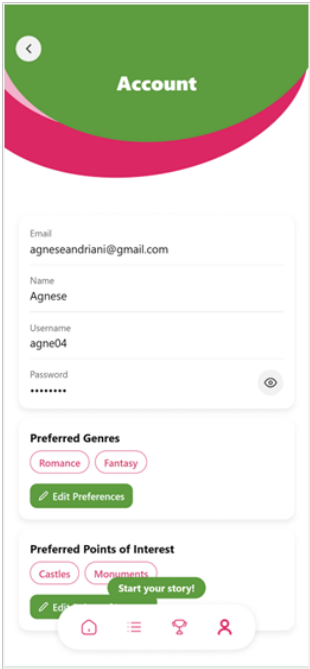


Figura 17: account

4 Tecnologie utilizzate

In questo capitolo vengono descritte le tecnologie utilizzate: vengono brevemente descritte e si spiega il loro utilizzo all'interno del progetto.

4.1 React Native

React Native [20] è un framework open-source per lo sviluppo di applicazioni mobile, mantenuto da Meta. Quello che React Native ci fornisce, è un insieme di componenti software riutilizzabili, librerie, strumenti e convenzioni che fornisce una base strutturata per la creazione di applicazioni. Il suo principale punto di forza è quello di permettere agli sviluppatori di scrivere codice una volta sola, in JavaScript o in TypeScript, ed eseguirlo sia su dispositivi iOS che Android, evitando la necessità di sviluppare due applicazioni distinte.

Grazie a React Native abbiamo potuto definire l'architettura generale dell'applicazione attraverso l'utilizzo di TypeScript. Un ulteriore vantaggio è la compatibilità con l'espansione Expo [11], un framework che estende React Native fornendo strumenti e libreria per semplificare lo sviluppo e la distribuzione dell'applicazione.

4.2 Expo Router

Expo Router [12] è una libreria che estende le funzionalità del framework Expo introducendo un sistema di navigazione basato sulla struttura delle cartelle, semplificando la gestione della navigazione.

Expo Router è stato utilizzato per implementare la navigazione tra le varie sezioni dell'applicazione, come la Home e l'Area Personale. Grazie alla sua struttura modulare, è stato possibile garantire un flusso di navigazione coerente, facilitando l'aggiunta di nuove sezioni o funzionalità.

4.3 TypeScript

TypeScript **TypeScriptDefinizione** è un linguaggio di programmazione open-source sviluppato da Microsoft che estende JavaScript introducendo il tipaggio statico. Questo significa che, a differenza di JavaScript, linguaggio debolmente tipizzato in cui il tipo delle variabili può cambiare durante l'esecuzione, TypeScript permette di dichiarare in anticipo il tipo di una variabile o di un parametro, rendendo il codice più leggibile e meno soggetto a errori **TypeScriptFunzionamento**.

TypeScript non viene eseguito direttamente dal browser, ma è un linguaggio compilato: il codice scritto in TypeScript viene trasformato in JavaScript standard, compatibile con qualsiasi ambiente che supporta ECMAScript. Questo approccio consente di mantenere tutti i vantaggi di JavaScript, aggiungendo al tempo stesso controlli statici che aiutano a individuare errori già in fase di sviluppo.

All'interno del progetto, TypeScript è stato utilizzato per implementare i componenti del frontend, ovvero le pagine dell'applicazione, in particolare la sua applicazione è stata fondamentale per definire lo stato dei componenti, tipizzare parametri di funzioni e gestire in modo sicuro il passaggio dei parametri tramite il router.

Infine, tramite l'oggetto StyleSheet [21], è stato possibile definire lo stile dei componenti in una forma simile al CSS, direttamente all'interno del codice TypeScript.

4.4 CSS in React Native

Il Cascading Style Sheet [7] è un linguaggio per la definizione dello stile delle pagine web, utilizzato in combinazione con HTML e JavaScript per separare la logica del contenuto dalla sua rappresentazione grafica. Nei progetti web tradizionali, lo stile viene definito tramite file `.css` esterni che contengono le regole di formattazione applicate agli elementi della pagina.

Tuttavia, in un progetto React Native come quello presentato, non si utilizzano i file CSS tradizionali: lo stile viene definito direttamente nel codice attraverso oggetti TypeScript che riprendono la sintassi del CSS. In particolare, è stato impiegato `StyleSheet`, una classe fornita dal framework che permette di creare e gestire gli stili in maniera ottimizzata.

L'utilizzo di `StyleSheet` consente di garantire la stessa visuale dell'applicazione sia su dispositivi mobili che su web. In questo progetto, è stato fondamentale per la definizione di layout responsive, colori e spaziatura delle diverse schermate.

4.5 Flask

Flask [27] è un micro-framework scritto in Python, pensato per applicazioni web. È basato sullo strumento WSGI (Web Server Gateway Interface), ovvero una specifica standard che definisce la comunicazione tra i server web e le applicazioni web in Python. È progettato per rendere l'avvio rapido e semplice, con la possibilità di scalare fino ad applicazioni complesse.

Nel progetto è stato scelto per l'implementazione del backend, che gestisce: le richieste provenienti dal frontend, le interrogazioni al database e la restituzione delle risposte in formato JSON. È stato utilizzato insieme all'estensione CORS (Cross-Origin Resource Sharing), un meccanismo di sicurezza che impedisce che le risorse vengano recuperate da un'origine diversa rispetto all'origine da cui è stata richiesta la risorsa [4].

4.6 MySQL

MySQL **MySQLFunzionamento** è un sistema di gestione di database relazionale (RDBMS). Un database relazionale **DatabaseRelazionale** è una raccolta organizzata di dati che utilizza tabelle per memorizzare informazioni correlate, con ogni riga che rappresenta un record e ogni colonna un attributo. Ogni riga della tabella è identificata da un ID univoco, chiamata chiave.

MySQL è stato utilizzato attraverso un modulo Python `db_connector.py` che contiene le funzioni per collegarsi al database, ci ha permesso di eseguire le query SQL e restituire i dati al backend Flask. È stato utilizzato per memorizzare i dati relativi all'utente, alle storie, ai punti di interesse, alle domande da porre al raggiungimento del punto e agli obiettivi.

4.7 Google Maps API

Le Google Maps JavaScript API [16] creano esperienze con mappe, posizioni dinamiche e interattive nelle applicazioni web. Nel progetto sono state utilizzate tramite la libreria `@react-google-maps/api`, che ha reso possibile la visualizzazione in tempo reale della posizione dell'utente sulla mappa e la rappresentazione della precisione del GPS tramite cerchi di accuratezza.

4.8 HTTP

La comunicazione tra il frontend e il backend viene tramite chiamate HTTP. L'HTTP (Hypertext Transfer Protocol) [8] è la base del World Wide Web e viene utilizzato per caricare le pagine web tramite collegamenti ipertestuali. HTTP è un protocollo a livello di applicazione progettato per trasferire informazioni tra dispositivi in una rete.

In particolare, viene utilizzata la funzione `fetch()` di JavaScript e TypeScript, che consente di inviare richieste al server e gestire le risposte in maniera asincrona. Le due modalità principali di chiamata adottate sono:

- **GET**: usata per recuperare dati (es. preferenze utente, statistiche settimanali, obiettivi);
- **POST**: usata per inviare dati al server (es. login, completamento di un punto, selezione di un genere, salvataggio di risposte).

Queste chiamate sono asincrone: ciò significa che l'applicazione non rimane bloccata in attesa della risposta del server, ma può continuare a funzionare in parallelo. Grazie all'integrazione con Flask, ogni endpoint backend restituisce una risposta in formato JSON, che viene interpretata dal frontend per aggiornare lo stato dei componenti e l'interfaccia utente.

4.9 JSON

JSON (JavaScript Object Notation) [22] non è un linguaggio di programmazione ma è una sintassi che viene utilizzata per scambiare dati semplicemente, indipendentemente dal linguaggio che si sta utilizzando.

Un oggetto JSON è formato da coppie nome-valore separate tra di loro dal carattere “:”, ogni riga che compone il JSON è separata da “,” e il tutto è racchiuso tra parentesi graffe.

Per gestire il JSON in TypeScript sono stati impiegati i metodi:

- `JSON.parse()`: per convertire una stringa JSON in un oggetto JavaScript utilizzabile dall'applicazione;
- `JSON.stringify()`: per trasformare un oggetto JavaScript in stringa JSON, da inviare nelle richieste HTTP.

Nel progetto, JSON è stato il formato principale utilizzato per lo scambio di informazioni tra client e server: il frontend invia richieste per trasmettere i dati, il backend Flask riceve e interpreta i dati e la risposta del server viene inviata in formato JSON grazie alla funzione `jsonify()`.

4.10 Librerie aggiuntive

Oltre alle tecnologie principali, sono state integrate alcune librerie per arricchire l'esperienza utente e migliorare la resa grafica dell'applicazione.

4.10.1 React Native Vector Icons

React Native Vector Icons [14] è una libreria che mette a disposizione un ampio set di icone personalizzabili, utilizzate nelle interfacce mobile moderne. Le icone sono vettoriali, quindi non perdono qualità quando ridimensionate, e possono essere facilmente integrate all'interno dei componenti React Native come pulsanti o menu.

Nel progetto questa libreria è stata utilizzata per arricchire la Bottom Navbar, la possibilità di tornare indietro o andare in avanti e i pulsanti di controllo audio, attraverso l'uso del set di icone Ionicons. Ciò ha permesso di rendere l'interfaccia più intuitiva, sfruttando simboli familiari come quello del volume, del play e della pausa.

4.10.2 AsyncStorage

AsyncStorage [28] è una libreria che fornisce un sistema di archiviazione locale basato su chiave-valore. Permette di salvare piccole quantità di dati in modo persistente sul dispositivo, anche dopo la chiusura dell'applicazione.

Nel progetto è stato impiegato per memorizzare le preferenze dell'utente, come i generi narrativi selezionati o i punti di interesse preferiti, e per gestire lo stato della sessione di navigazione. In questo modo, l'utente può riprendere la propria esperienza anche dopo aver chiuso e riaperto l'applicazione.

4.10.3 React Native Responsive Screen

React Native Responsive Screen [3] è una libreria che semplifica l'adattamento delle interfacce grafiche a diversi dispositivi e risoluzioni. Essa fornisce funzioni utili per definire dimensioni relative alla larghezza o all'altezza

dello schermo, garantendo un design responsivo sia su smartphone di piccole dimensioni che su tablet.

L'applicazione l'ha sfruttata per rendere le schermate leggibili e ben proporzionate su più dispositivi, senza necessità di creare layout specifici per ogni modello.

4.10.4 Expo Speech e Web SpeechSynthesis API

Per la lettura dei testi delle storie è stata sfruttata la Web SpeechSynthesis API e il pacchetto Expo Speech [13]. Queste tecnologie hanno reso possibile l'integrazione della sintesi vocale (Text-to-Speech), trasformando la narrazione scritta in audio.

La libreria è stata utilizzata per migliorare l'accessibilità dell'applicazione e offrire un'esperienza più immersiva, consentendo all'utente di ascoltare la storia mentre percorre il tragitto.

5 Dettagli tecnici

Questo capitolo descrive i dettagli tecnici e implementativi dell'applicazione sviluppata. Vengono illustrate l'architettura software, il modello dell'utente, la formalizzazione delle preferenze, l'organizzazione del codice in moduli, la struttura del database e vengono riportati frammenti di codice significativi per comprendere il funzionamento dell'interfaccia e delle API.

5.1 Architettura software e modello SPA

Il progetto è stato realizzato seguendo l'architettura Single Page Application (SPA) [23]. In una SPA, l'interfaccia utente viene caricata una sola volta, e le interazioni successive avvengono tramite chiamate asincrone verso il server, solitamente allo scatenarsi di determinati eventi generati dall'utente.

In particolare, l'applicazione utilizza l'API fetch di JavaScript [9] per effettuare richieste asincrone al backend sviluppato in Flask. I dati vengono scambiati in formato JSON, che costituisce lo standard per la comunicazione client-server.

La Figura 18 mostra un'architettura a servizi REST. La RESTful API [6], è un'interfaccia utilizzata da due sistemi per scambiare informazioni in modo sicuro, seguendo uno standard di comunicazione software sicuro, affidabile ed efficiente. In particolare, REST (Representational State Transfer) è l'architettura software che impone queste condizioni sul funzionamento dell'API. Diversi client, come applicazioni mobile, web o desktop, comunicano con un'unica interfaccia RESTful che gestisce la logica di business e si collega a un database relazionale. Questo schema è perfettamente rappresentativo del

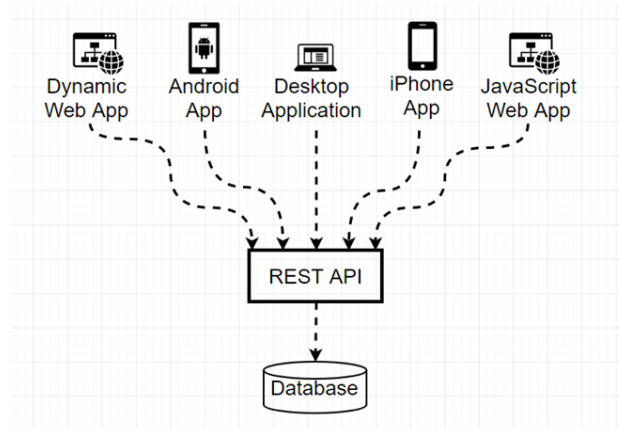


Figura 18: schema tipico dell'architettura RESTful

progetto sviluppato, dove la SPA realizzata in React Native comunica con un backend Flask attraverso chiamate HTTP fetch, e i dati vengono salvati o recuperati da un database MySQL.

Ad esempio, una richiesta di login viene inviata al server tramite fetch, il quale risponde con un JSON che contiene l'esito dell'autenticazione e i dati dell'utente:

```

const response = await fetch('http://127.0.0.1:5000/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
const data = await response.json();
  
```

Il client elabora la risposta e aggiorna lo stato dell'applicazione in tempo reale, senza interruzioni.

5.2 Modello dell'utente

Il modello dell'utente comprende i campi principali di autenticazione e i dati relativi al gioco come chilometri percorsi, calorie consumate e passi effettuati. La creazione di un nuovo utente avviene attraverso l'endpoint `/add-user`, che

salva i dati nel database MySQL. Nel codice del server, la route chiamerà la funzione `insert_user`, la quale si occuperà di inserire i primi dati relativi al nuovo utente. Vediamo il codice:

```
@app.route('/add-user', methods=['POST', 'OPTIONS'],
    ↪ strict_slashes=False)
@handle_cors
def add_user():
    try:
        data = request.get_json(force=True)
        user_data = data.get("user")
        user = db_connector.insert_user(user_data)
        return jsonify(user)
    except Exception as e:
        print(e)
        return jsonify({'error': 'Internal server error'}), 500

def insert_user(user_data):
    cnx = create_db_connection()
    cursor = cnx.cursor()
    try:
        insert_query = """INSERT INTO users (name, username, email,
    ↪ password, total_km, total_calories, total_steps )
    ↪ VALUES (%s, %s, %s, %s, %s, %s, %s)"""

        cursor.execute(insert_query, (user_data['name'],
    ↪ user_data['username'], user_data['email'],
    ↪ user_data['password'], user_data['total_km'],
    ↪ user_data['total_calories'], user_data['total_steps']))
        cnx.commit()
        print(f'MySQL: inserted with ID={cursor.lastrowid} ')
        return {'id': cursor.lastrowid, **user_data}
    except mysql.connector.Error as error:
        print("Failed to insert into MySQL table
    ↪ {}".format(error))
```

```

finally:
    if cnx.is_connected():
        cursor.close()
        cnx.close()
        print("MySQL connection is closed")
    
```

5.3 Formalizzazione delle preferenze

Le preferenze dell'utente sono divise in due categorie principali: Generi narrativi: Fantasy, History, Adventure,... Punti di Interesse: Castles, Monuments,... Ogni preferenza viene salvata nel database e associata allo user_id. La scelta avviene tramite schermate interattive che permettono di selezionare più opzioni. Esempio di invio delle preferenze verso il server:

```

const response = await
↪ fetch('http://127.0.0.1:5000/preferences/adding', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        user_id: userId,
        genres: selectedGenres,
    }),
});
const data = await response.json();

const response = await
↪ fetch('http://127.0.0.1:5000/pointOfInterest/adding', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        user_id: userId,
        point_of_interest: selectedPOI,
    }),
    
```

```
});
const data = await response.json();
```

Le preferenze vengono legate all'utente corretto e si hanno due chiamate differenti per generi e punti di interesse. Inoltre, per il recupero delle preferenze di ciascun utente verrà usata una chiamata fetch con lo user_id come parametro di query, ad esempio:

```
const API_BASE = 'http://127.0.0.1:5000';
const [uRes, pRes] = await Promise.all([
    fetch(`${API_BASE}/test-get-user?user_id=${stored.id}`),
    fetch(`${API_BASE}/preferences?user_id=${stored.id}`),
]);
```

L'utilizzo di Promise.all ci permette di recuperare contemporaneamente i dati relativi alla registrazione dell'utente e alle sue preferenze; questa soluzione ci consente di minimizzare i tempi di attesa poiché le due richieste vengono effettuate in parallelo. Nel backend avremo il recupero delle informazioni in questo modo:

```
def get_preferences_by_user(user_id):
    conn = create_db_connection()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute(
            "SELECT id, user_id, genre, point_of_interest FROM
            → preferences WHERE user_id = %s ORDER BY id ASC",
            (user_id,)
        )
        rows = cursor.fetchall()
        return rows
    except mysql.connector.Error as error:
        print(f"Errore MySQL get_preferences_by_user: {error}")
        return []
    finally:
```

```
if conn.is_connected():  
    cursor.close()  
    conn.close()
```

5.4 Organizzazione del codice

Per lo sviluppo dell'applicazione è stata adottata una struttura modulare. Ogni funzionalità è stata suddivisa in file distinti in modo da rendere il codice più ordinato, facilmente manutenibile e riutilizzabile. Per il frontend:

- /time.tsx → selezione della durata dell'attività
- /places.tsx → inserimento facoltativo dei luoghi
- /genre.tsx → scelta dei generi narrativi
- /story.tsx e /points.tsx → visualizzazione della mappa e interazione con i contenuti
- /account.tsx, /preferences.tsx, /pointsOfInterest.tsx → gestione e modifica del profilo

Nel backend sono stati inseriti:

- server.py → definizione degli endpoint REST
- db_connector.py → gestione connessione MySQL
- docker-init.sh → script per l'inizializzazione del container MySQL

Questa suddivisione permette di avere un'architettura pulita e ben organizzata: ogni file è responsabile di un compito specifico e le dipendenze sono ridotte al minimo. In questo modo è possibile estendere l'applicazione o modificarne le funzionalità senza compromettere l'intero progetto.

5.5 Codice rilevante

In questo paragrafo vengono presentate delle porzioni di codice rilevante, ovvero la rappresentazione delle funzionalità descritte nei capitoli precedenti.

5.5.1 Costruzione e gestione dei chunk di testo

Quando viene caricata una storia, il testo (titolo, introduzione e tema) viene suddiviso in “chunk” per la lettura progressiva tramite TTS.

```
function splitIntoChunks(text: string) {
  if (!text) return [];
  const sentences = text.split(/(?<=[.!?])\s+/); // suddividere in
    ↪ frasi
  const MAX = 260;
  const packed: string[] = [];
  for (const s of sentences) {
    if (!s) continue;
    if (!packed.length) packed.push(s);
    else {
      const last = packed[packed.length - 1];
      if ((last + ' ' + s).length <= MAX) packed[packed.length -
        ↪ 1] = last + ' ' + s;
      else packed.push(s);
    }
  }
  return packed;
}
```

Questa funzione ha il compito di prendere un testo narrativo esteso (composto da titolo, introduzione e tema della storia) e suddividerlo in porzioni più piccole, chiamate chunk. Il criterio di divisione principale è la punteggiatura forte (. ! ?), che corrisponde alla fine di una frase. Inoltre, per evitare che il sintetizzatore vocale (Text-to-Speech) riceva stringhe troppo lunghe da leggere in un’unica volta, viene imposto un limite massimo (MAX = 260 ca-

ratteri). Se concatenando due frasi si supera questa soglia, la funzione inizia un nuovo chunk. In questo modo si ottengono segmenti di testo ottimizzati per la lettura ad alta voce: abbastanza brevi da non risultare pesanti per l'utente e abbastanza lunghi da mantenere la fluidità della narrazione. Senza questa suddivisione, l'esperienza vocale sarebbe meno chiara e rischierebbe di perdere l'attenzione dell'ascoltatore.

5.5.2 Lettura dei chunk tramite TTS

La funzione `speakChunkAt` gestisce la lettura vocale di un chunk, con controlli su stato (stop/continue) e volume.

```
function speakChunkAt(i: number, forceVolume?: number) {
  const synth = synthRef.current;
  const chunks = chunksRef.current;
  if (!synth || !chunks?.length) return;
  if (i < 0 || i >= chunks.length) return;

  synth.cancel(); // interrompe la lettura precedente

  const u = new SpeechSynthesisUtterance(chunks[i]);
  u.lang = 'en-GB';
  u.rate = 1;
  u.pitch = 1;
  u.volume = typeof forceVolume === 'number' ? forceVolume :
    ↪ (isAudioPlayingRef.current ? 1 : 0);

  u.onend = () => {
    if (storyStateRef.current === 'continue') {
      idxRef.current = i + 1;
      if (idxRef.current < chunks.length)
        ↪ speakChunkAt(idxRef.current);
    }
  };
};
```

```
synth.speak(u);
}
```

Questa funzione ha il compito di leggere con voce sintetizzata un chunk specifico (identificato dall'indice *i*). Prima di avviare la lettura, viene invocato `synth.cancel()` per interrompere eventuali letture già in corso: ciò garantisce che non si verifichino sovrapposizioni o eco indesiderati. Il chunk selezionato viene quindi trasformato in un oggetto `SpeechSynthesisUtterance`, al quale vengono configurati parametri fondamentali per la resa vocale:

- la lingua (`en-GB`),
- la velocità (`rate`),
- l'intonazione (`pitch`),
- il volume (`volume`).

Particolarmente interessante è la gestione dello stato: grazie all'evento `onend`, quando un chunk termina di essere letto, la funzione controlla se lo stato della storia è ancora impostato su `continue`. In tal caso, viene incrementato l'indice e avviata automaticamente la lettura del chunk successivo. Questo meccanismo permette di implementare un flusso narrativo continuo, che può essere interrotto o ripreso in qualsiasi momento. È una logica simile a un lettore multimediale che, dopo aver terminato una traccia, passa automaticamente alla successiva.

5.5.3 Costruzione del testo con descrizioni e domande (`points.tsx`)

Quando l'utente raggiunge un punto di interesse, viene costruito un testo che unisce la descrizione del punto e le domande a esso associate.

```
function buildPointSpeech(point: any, qlist: any[]) {
  const parts: string[] = [];
  if (point?.text) parts.push(String(point.text)); // descrizione

  qlist.forEach((q) => {
    if (q?.question) parts.push(String(q.question));
    const answers = [q.first_answer, q.second_answer,
      ↪ q.third_answer].filter(Boolean);
    answers.forEach((a, idx) => {
      const letter = String.fromCharCode(65 + idx); // A/B/C
      parts.push(`ℓ{letter}. ℓ{a} `);
    });
  });

  return parts.join(' ');
}
```

Questa funzione costruisce il testo che verrà letto dal TTS quando l'utente si trova in prossimità di un punto di interesse. Non si limita alla descrizione del luogo (`point.text`), ma integra anche le domande e le possibili risposte multiple. Per ogni domanda viene inserito nel testo sia il quesito che le alternative di risposta. Queste ultime vengono etichettate con lettere dell'alfabeto (A, B, C), esattamente come in un quiz a scelta multipla. Il risultato finale è un testo concatenato che unisce:

- la descrizione narrativa del luogo,
- le domande legate al punto,
- le possibili risposte.

In questo modo l'utente riceve un'esperienza interattiva completamente vocale: può ascoltare la storia e contemporaneamente affrontare quiz o domande di approfondimento senza dover necessariamente guardare lo schermo.

5.5.4 Attivazione automatica del TTS ai punti di interesse

Quando l'utente si avvicina a un punto non ancora completato, il testo relativo viene automaticamente letto ad alta voce.

```
useEffect(() => {
  if (!nextVisiblePoint || navbarState !== 'continue') return;

  const alreadyRead = lastReadPointIdRef.current ===
    ↪ Number(nextVisiblePoint.point_id);
  const relatedQuestions = questions.filter(
    (q) => Number(q.point_id) ===
    ↪ Number(nextVisiblePoint.point_id)
  );

  if (!alreadyRead || !synthRef.current?.speaking) {
    const speech = buildPointSpeech(nextVisiblePoint,
    ↪ relatedQuestions);
    lastReadPointIdRef.current =
    ↪ Number(nextVisiblePoint.point_id);
    speakText(speech);
  }
}, [nextVisiblePoint, navbarState, questions]);
```

In questa porzione di codice controlliamo continuamente la posizione dell'utente rispetto ai punti narrativi presenti sulla mappa. Quando viene rilevato un nuovo punto entro il raggio definito, e se lo stato della navbar è impostato su *continue*, viene attivata automaticamente la lettura vocale del contenuto relativo a quel punto. Il controllo `alreadyRead` serve a evitare che lo stesso punto venga letto più volte: viene infatti memorizzato l'ultimo `point_id` già pronunciato. Se il punto è nuovo, viene richiamata la funzione `buildPointSpeech`, che genera il testo unendo descrizione e domande, e subito dopo viene invocata `speakText` per avviare la lettura vocale. Questo meccanismo lega in modo diretto la geolocalizzazione alla narrazione inte-

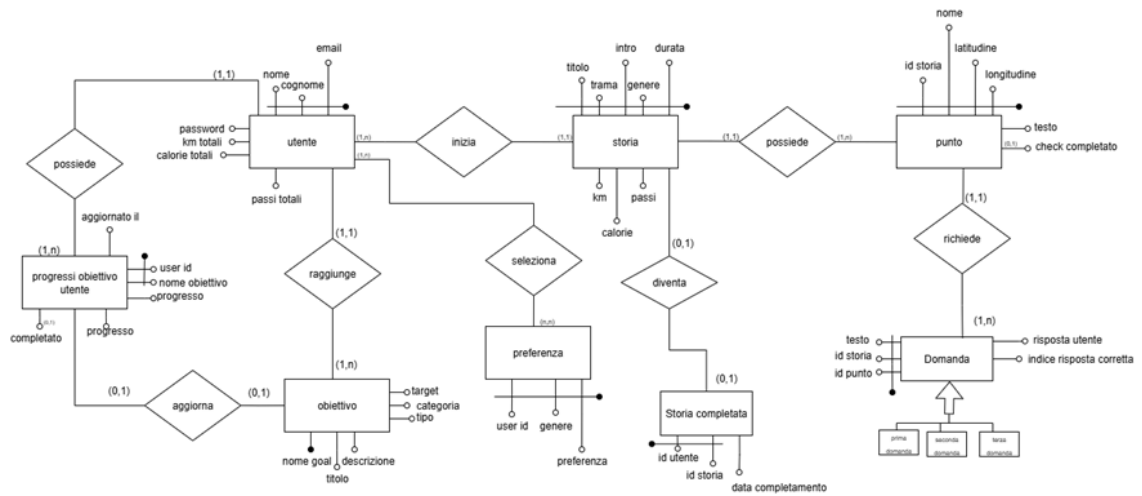


Figura 19: schema ER

rattiva: il contenuto non viene fruito in modo lineare, ma viene sbloccato e raccontato dinamicamente solo quando l'utente si avvicina fisicamente al luogo corrispondente.

5.6 Database

Per la persistenza dei dati è stato scelto MySQL [24], un DBMS relazionale open source ampiamente utilizzato per applicazioni web e mobile. MySQL è facilmente integrabile con il backend scritto in Python di cui facciamo utilizzo. Il database è stato predisposto tramite lo script `docker-init-w.sh`, che avvia un container Docker con MySQL già configurato. Questo approccio ha consentito di semplificare il setup del progetto e di garantire la portabilità dell'ambiente di sviluppo, riducendo i problemi di compatibilità tra diverse macchine.

5.6.1 Le tabelle

Le tabelle utilizzate dall'applicazione sono:

- `users`: contiene i dati relativi agli utenti registrati, sia anagrafici sia di progresso iniziale. Questi ultimi verranno posti a zero al momento della registrazione e verranno aggiornati in una tabella a parte.
- `goals`: presenta un elenco di obiettivi generali che ogni utente può raggiungere all'interno dell'applicazione.
- `points`: presenta tutti i dati relativi ai punti di ogni storia come il testo, le coordinate e se il punto è stato completato o meno.
- `preferences`: vengono salvate le preferenze relative a generi narrativi e a punti di interesse per ogni utente.
- `questions`: vengono registrate le domande che verranno poste al raggiungimento degli obiettivi. Inoltre, vengono inserite le possibili risposte. Queste vengono enumerate in modo tale che si possa controllare se l'utente ha dato la risposta giusta.
- `story`: La tabella presenta tutte le caratteristiche relative alla storia, come titolo, intro, trama e genere.
- `story_completed`: per ogni utente, viene registrata la data di completamento di una storia.
- `user_goal_progress`: questa è la tabella relativa al completamento degli obiettivi generali: l'utente sarà in grado di vedere il completamento o l'avanzamento di obiettivo proposto dall'applicazione.

5.6.2 Il funzionamento

Le operazioni sul database sono state incapsulate nel modulo `db.connector.py`, che si occupa di aprire la connessione, eseguire le query e restituire i risultati. Il backend Flask invoca le funzioni definite in `db.connector.py`, per ottenere o salvare i dati. Queste funzioni gestiscono in autonomia apertura e chiusura della connessione, riducendo al minimo il rischio di errori. Il metodo `cursor.execute()` è quello che effettivamente inoltra la query al DBMS. Può

essere usato sia per query di lettura (SELECT) sia per query di scrittura (INSERT, UPDATE, DELETE). Ecco un esempio:

```
def get_story_by_genre(genre):
    conn = create_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM story WHERE LOWER(genre) =
    ↪ LOWER(%s) LIMIT 1", (genre,))
    story = cursor.fetchone()
    cursor.close()
    conn.close()
    print(story)
    return story
```

Per semplificare la gestione del database durante lo sviluppo è stato impiegato Docker Desktop [10]. Docker consente di eseguire MySQL in un container isolato, evitando la necessità di installarlo e configurarlo manualmente sul sistema operativo host. L'uso di Docker garantisce la portabilità, infatti uno stesso container può essere eseguito su diversi sistemi operativi e il database può essere facilmente resettato o ricreato in caso di problemi, eliminando e ricreando il container.

6 Conclusioni e Sviluppi Futuri

Il progetto presentato in questa tesi ha mirato alla realizzazione di una prima versione funzionante dell'applicazione mobile che possa combinare gamification, narrazione interattiva e strumenti di geolocalizzazione per guidare l'esperienza degli utenti. È stata sviluppata la visualizzazione del flusso narrativo e delle preferenze dell'utente, permettendogli di definire la quantità di tempo a disposizione e fornendogli una narrazione personalizzata. La descrizione dei punti di interesse e delle storie sono stati arricchiti con funzionalità di sintesi locale. L'applicazione fornisce un insieme di funzionalità che collegano l'esperienza narrativa con l'attività fisica, il tutto in un'esperienza unica e personalizzata.

I possibili sviluppi futuri sono:

- In questa tesi non ci siamo occupati di realizzare un vero e proprio sistema che generasse attivamente delle storie basate sulle preferenze dell'utente, ma è stato fornito un esempio per dimostrare il funzionamento generale dell'applicazione. Uno sviluppo futuro è quello di arricchire l'applicazione con un sistema che sappia generare tali storie, domande e punti di interesse.
- Le storie e i punti di interesse potrebbero essere arricchiti con immagini e video provenienti da fonti storiche e artistiche certe.
- Un ulteriore sviluppo futuro potrebbe riguardare l'aumento della gamification all'interno del progetto, dando la possibilità non solo di ri-

spondere alle domande, ma anche di risolvere puzzle o enigmi di vario tipo.

- Migliorare l'accessibilità dando la possibilità all'utente di modificare la velocità della sintesi vocale, utilizzando anche interfacce ad alto contrasto per utenti ipovedenti.
- Realizzare una community che permetta all'utente di inserirsi all'interno di una classifica globale attraverso l'utilizzo dell'applicazione e al sempre più alto numero di storie completate, facendo in modo che più utenti entrino in contatto tra di loro, uniti da passioni comuni.

In conclusione, l'applicazione dimostra come le tecnologie moderne possano contribuire a realizzare esperienze innovative e interattive, non solo dal punto di vista artistico, ma anche da un punto di vista culturale più ampio. L'evoluzione del progetto potrà poi renderlo uno strumento di svago, ma anche un mezzo educativo e sportivo, in modo tale che si adatti ai vari contesti e necessità.

Bibliografia

- [1] Niantic. “Pokémon Go.” Consultato nel 2025. (2016), indirizzo: <https://pokemongo.com/it>.
- [2] ESRI Community. “Distance on a sphere: the Haversine formula.” Consultato nel 2025. (2023), indirizzo: <https://community.esri.com/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128>.
- [3] A. Omuiz. “Responsive Design in React Native: Building Apps for Multiple Screen Sizes.” Consultato nel 2025. (2023), indirizzo: <https://dev.to/aomuiz/responsive-design-in-react-native-building-apps-for-multiple-screen-sizes-1fnf>.
- [4] M. Terrano. “CORS in a Flask API.” Consultato nel 2025. (2023), indirizzo: <https://medium.com/@mterrano1/cors-in-a-flask-api-38051388f8cc>.
- [5] AIRC. “Sedentarietà e cancro.” Consultato nel 2025. (2024), indirizzo: <https://www.airc.it/cancro/prevenzione-tumore/attivita-fisica/sedentarieta-e-cancro#:~:text=Lo%20scarso%20movimento%20aumenta%20infatti,depressione%2C%20e%20anche%20il%20cancro..>
- [6] Amazon Web Services. “Cos’è un’API RESTful.” Consultato nel 2025. (2024), indirizzo: <https://aws.amazon.com/it/what-is/restful-api/#:~:text=di%20API%20RESTful?-,Cos%20%27%20API%20RESTful?,software%20sicuri%2C%20affidabili%20ed%20efficienti..>

- [7] Boolean Careers. “CSS: cos’è, come funziona e a cosa serve.” Consultato nel 2025. (2024), indirizzo: <https://boolean.careers/blog/css-cose-come-funziona-e-a-cosa-serve>.
- [8] Cloudflare. “Cos’è l’HTTP.” Consultato nel 2025. (2024), indirizzo: <https://www.cloudflare.com/it-it/learning/ddos/glossary/hypertext-transfer-protocol-http/>.
- [9] DigitalOcean. “How to Use the JavaScript Fetch API to Get Data.” Consultato nel 2025. (2024), indirizzo: <https://www.digitalocean.com/community/tutorials/how-to-use-the-javascript-fetch-api-to-get-data>.
- [10] Docker Inc. “Docker Desktop Documentation.” Consultato nel 2025. (2024), indirizzo: <https://docs.docker.com/desktop/>.
- [11] Expo. “Expo Official Documentation.” Consultato nel 2025. (2024), indirizzo: <https://expo.dev/>.
- [12] Expo. “Expo Router Introduction.” Consultato nel 2025. (2024), indirizzo: <https://docs.expo.dev/router/introduction/>.
- [13] Expo. “Text To Speech SDK.” Consultato nel 2025. (2024), indirizzo: <https://docs.expo.dev/versions/latest/sdk/speech/>.
- [14] Expo Icons. “Expo Icon Library.” Consultato nel 2025. (2024), indirizzo: <https://icons.expo.fyi/Index>.
- [15] Google Cloud. “MySQL Overview.” Consultato nel 2025. (2024), indirizzo: <https://cloud.google.com/mysql?hl=it>.
- [16] Google Developers. “Maps JavaScript API.” Consultato nel 2025. (2024), indirizzo: <https://developers.google.com/maps/documentation/javascript?hl=it>.
- [17] GuideXpress. “Scopri l’app per visitare città che sta rivoluzionando il modo di viaggiare.” Consultato nel 2025. (2024), indirizzo: <https://app.guidexpress.com/it/scopri-lapp-per-visitare-citta-che-sta-rivoluzionando-il-modo-di-viaggiare/#:~:text=>

- Con % 20Guidexpress % 2C % 20viaggiare % 20non % 20 % C3 % A8 , con % 20Guidexpress%20un'esperienza%20indimenticabile..
- [18] IBM. "Cosa sono le API." Consultato nel 2025. (2024), indirizzo: <https://www.ibm.com/it-it/think/topics/api>.
 - [19] izi.TRAVEL. "Chi siamo." Consultato nel 2025. (2024), indirizzo: <https://izi.travel/it/chi-siamo>.
 - [20] Meta. "React Native." Consultato nel 2025. (2024), indirizzo: <https://reactnative.dev/>.
 - [21] Meta. "React Native Stylesheet." Consultato nel 2025. (2024), indirizzo: <https://reactnative.dev/docs/0.79/stylesheet>.
 - [22] Mozilla Developer Network. "JSON Reference." Consultato nel 2025. (2024), indirizzo: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON.
 - [23] Mozilla Developer Network. "Single Page Application." Consultato nel 2025. (2024), indirizzo: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
 - [24] MySQL. "Cos'è MySQL." Consultato nel 2025. (2024), indirizzo: <https://www.mysql.com/it/>.
 - [25] Niantic Labs. "Niantic Social Impact." Consultato nel 2025. (2024), indirizzo: <https://nianticlabs.com/social-impact?hl=it>.
 - [26] Oracle. "Cos'è un database relazionale." Consultato nel 2025. (2024), indirizzo: <https://www.oracle.com/it/database/what-is-a-relational-database/>.
 - [27] Pallets Projects. "Flask Documentation." Consultato nel 2025. (2024), indirizzo: <https://flask.palletsprojects.com/en/stable/>.
 - [28] React Native. "Async Storage." Consultato nel 2025. (2024), indirizzo: <https://reactnative.dev/docs/asyncstorage>.
 - [29] TypeScript. "TypeScript Official Documentation." Consultato nel 2025. (2024), indirizzo: <https://www.typescriptlang.org/>.

- [30] W3Schools. “TypeScript Introduction.” Consultato nel 2025. (2024), indirizzo: https://www.w3schools.com/typescript/typescript_intro.php.